

这份讲义 (Lecture 13) 的内容非常循序渐进, 它像讲故事一样带我们从一个最笨的办法开始, 一步步解决遇到的问题, 最后推导出完美的哈希表。我将为你补充讲义中非常具体的数据、例子和数学原理:

1. 为什么我们需要哈希表? (从 $O(\log N)$ 到 $O(1)$)

在学习哈希表之前, 我们学过的最快的数据结构是红黑树 (Red-Black Trees) 和二叉搜索树 (BST), 它们的增删改查时间复杂度最好也就是 $O(\log N)$ 。而哈希表的目标是追求极致的 $O(1)$ (常数时间)。

初代构想与失败:

- **INTSet (用数组直接存):** 假设我们要存数字, 最简单的办法是建一个巨大的布尔数组 `boolean`。存入数字 5, 就把 `items[5] = true`。这确实是 $O(1)$, 但极其浪费内存, 且只能存正整数。
- **ENGSet (存英文):** 如果要存英文字符串 (比如 `cat`), 我们可以把字符串当成 27 进制 (26 个字母+1) 转换为数字。例如 $cat = c\$ \times 27^2\$ + a\$ \times 27^1\$ + t\$ \times 27^0\$$ 。
- **Unicode (存中文):** 如果要存中文, 比如“守门员”, 中文字符的 Unicode 最大值约 40959, 那么计算出来的值会是 $23432\$ \times 40959^2\$ + 38376\$ \times 40959^1\$ + 21592\$ \times 40959^0\$ = 39,312,024,869,368$ 。
- **致命问题 (Integer Overflow):** Java 中的 `int` 最大只能表示约 21.47 亿 (2,147,483,647)。上面的 39 万亿 远超这个数字, 会导致 **整型溢出**, 变成负数, 从而导致数组越界报错。

2. 无法避免的宿命: 哈希冲突 (Collisions)

- **鸽巢原理 (Pigeonhole Principle):** 讲义用鸽子和鸽巢举例 (9 个巢放 10 只鸽子, 必然有巢里有 2 只)。在 Java 中, 32 位整数 (`int`) 最多只能表示约 42.9 亿 个不同的哈希码。但是宇宙中的星球数量、或者世界上所有可能的字符串组合, 远超 42.9 亿。
- **结论:** 无论你的算法多完美, 必定会出现不同的字符串计算出完全一样的哈希码。冲突是绝对不可避免的。

3. 处理冲突的具体细节: 分离链接法 (Separate Chaining)

既然冲突无法避免, 假设字符串 `"bee"` 和 `"w@$p"` 计算出的索引都是 718, 该怎么办?

- **桶 (Bucket):** 我们不再让数组里直接存 `true/false`, 而是存一个 **链表 (Linked List)** (或者 `ArrayList`)。这个链表被称为“桶”。

- 当 "bee" 存入时，索引 718 的链表放入 "bee"；当 "w@p" 存入时，顺着加到索引 718 的链表后面。
- **性能隐患：** 如果我们固定只有 5 个桶 ($M=5$)，但存入了 N 个元素。即使运气极好、分布极其均匀，每个链表的长度 Q 也会变成 $N/5$ 。在最坏情况下（所有元素全挤在一个桶里），链表长度 Q 会等于 N 。此时，查找和插入的时间复杂度会退化成**线性时间 $O(N)$** ，这就违背了我们追求 $O(1)$ 的初衷。

4. 解决性能退化：动态扩容机制 (Resizing)

为了保证链表长度 Q 永远是常数 ($O(1)$)，我们需要引入一个指标并制定扩容规则：

- **负载因子 (Load Factor)：** 定义为 N/M （元素总数 $\div$ 桶的总数），它代表了哈希表的拥挤程度。
- **扩容触发条件：** 讲义中给出的标准是，当 $N/M \geq 1.5$ 时，触发动态扩容，将桶的数量 M **翻倍 (Double)**。
- **具体过程演示：** 假设我们初始有 4 个桶 ($M=4$)。随着元素不断添加，当加到第 6 个元素时 ($N=6$)， $N/M = 1.5$ 。此时哈希表会立刻创建一个 $M=8$ 的新数组，并将这 6 个元素**重新计算索引并重新分配** (Redistribute) 到这 8 个新桶里，扩容后负载因子降回 0.75。
- **摊还分析 (Amortized Analysis)：** 虽然每次扩容时，重新分配所有元素需要消耗 $O(N)$ 的时间，但由于我们是按倍数（乘法）扩容，扩容不会频繁发生。平摊到每一次操作上，平均运行时间依然被保证为 **$O(1)$** 。

5. 决定生死的核心：如何设计出色的哈希函数？

扩容机制能保证整体空间足够，但这有个前提：**元素必须能被均匀地分布 (Evenly Distributed)** 到各个桶里。如果哈希函数设计得不好，元素全挤在一个桶里，扩容也没用。

- **避开致命的偶数基数 (Base 126 的教训)：** 如果你用 126 作为基数来计算字符串哈希，因为 126 是偶数，在 Java 中 126^{32} 不断相乘并溢出后，**结果会精确地变成 0**。这会导致一个灾难性后果：像 "Laughter is the best thing on the earth." 和 "Muesli is the best thing on the earth." 这样前缀不同、但**最后 32 个字符完全一样的句子，计算出的哈希码竟然完全一样**（都是 634199182），这将引发极为严重的冲突。
- **采用小素数 (Small Prime)：** 为了解决上面的问题，Java 的 String.hashCode() 默认选择 **31** 这个小素数作为乘法基数。

- **为什么是素数？** 素数永远不是偶数，多次相乘溢出后不会归零，这打破了上述的魔咒。
- **为什么是小数字？** 小数字计算起来成本更低，速度更快。
- **处理负数的终极防线：** 计算出的 `hashCode()` 可能是负数。如果你用 `hash % M` 取模来找桶的位置，负数取模在 Java 中依然是负数，会导致数组索引越界报错。因此，最终定位必须使用 **`Math.floorMod(hashCode, M)`**，它能确保把任意正负的哈希码安全地转换为 `0` 到 `M-1` 的合法正数索引。

1. 为什么我们需要哈希表？（从 $O(\log N)$ 到 $O(1)$ ）

在学习哈希表之前，我们学过的最快的数据结构是红黑树（Red-Black Trees）和二叉搜索树（BST），它们的增删改查时间复杂度最好也就是 $O(\log N)$ ¹。而哈希表的目标是追求极致的 $O(1)$ （常数时间）¹。

初代构想与失败：

- **INTSet（用数组直接存）：** 假设我们要存数字，最简单的办法是建一个巨大的布尔数组 `boolean`。存入数字 `5`，就把 `items[5] = true`。这确实是 $O(1)$ ，但极其浪费内存，且只能存正整数 ³。
- **ENGSet（存英文）：** 如果要存英文字符串（比如 `cat`），我们可以把字符串当成 27 进制（26 个字母+1）转换为数字。例如 `cat = c × 272 + a × 271 + t × 270` ⁴。
- **Unicode（存中文）：** 如果要存中文，比如“守门员”，中文字符的 Unicode 最大值约 40959，那么计算出来的值会是 `23432 × 409592 + 38376 × 409591 + 21592 × 409590 = 39,312,024,869,368` ⁵ ⁶。
- **致命问题（Integer Overflow）：** Java 中的 `int` 最大只能表示约 **21.47亿**（2,147,483,647）。上面的 39万亿 远超这个数字，会导致 **整型溢出**，变成负数，从而导致数组越界报错 ⁶。

2. 无法避免的宿命：哈希冲突 (Collisions)

- **鸽巢原理 (Pigeonhole Principle)：** 讲义用鸽子和鸽巢举例（9个巢放10只鸽子，必然有巢里有2只）。在 Java 中，32位整数（`int`）最多只能表示约 **42.9亿** 个不同的哈希码。但是宇宙中的星球数量、或者世界上所有可能的字符串组合，远超 42.9亿 ⁷。
- **结论：** 无论你的算法多完美，必定会出现不同的字符串计算出完全一样的哈希码。冲突是绝对不可避免的 ⁷。

3. 处理冲突的具体细节：分离链接法 (Separate Chaining)

既然冲突无法避免，假设字符串 "bee" 和 "w@p" 计算出的索引都是 718，该怎么办⁸？

- **桶 (Bucket)**：我们不再让数组里直接存 true/false，而是存一个链表 (Linked List)（或者 ArrayList）。这个链表被称为“桶”⁸。
- 当 "bee" 存入时，索引 718 的链表放入 "bee"；当 "w@p" 存入时，顺着加到索引 718 的链表后面⁸。
- **性能隐患**：如果我们固定只有 5 个桶 ($M = 5$)，但存入了 N 个元素。即使运气极好、分布极其均匀，每个链表的长度 Q 也会变成 $N/5$ 。在最坏情况下（所有元素全挤在一个桶里），链表长度 Q 会等于 N 。此时，查找和插入的时间复杂度会退化到线性时间 $O(N)$ ，这就违背了我们追求 $O(1)$ 的初衷⁹。

4. 解决性能退化：动态扩容机制 (Resizing)

为了保证链表长度 Q 永远是常数 ($O(1)$)，我们需要引入一个指标并制定扩容规则：

- **负载因子 (Load Factor)**：定义为 N/M （元素总数 ÷ 桶的总数），它代表了哈希表的拥挤程度¹¹。
- **扩容触发条件**：讲义中给出的标准是，当 $N/M \geq 1.5$ 时，触发动态扩容，将桶的数量 M 翻倍 (Double)^{11 12}。
- **具体过程演示**：假设我们初始有 4 个桶 ($M = 4$)。随着元素不断添加，当加到第 6 个元素时 ($N = 6$)， $N/M = 1.5$ 。此时哈希表会立刻创建一个 $M = 8$ 的新数组，并将这 6 个元素重新计算索引并重新分配 (Redistribute) 到这 8 个新桶里，扩容后负载因子降回 0.75¹³。
- **摊还分析 (Amortized Analysis)**：虽然每次扩容时，重新分配所有元素需要消耗 $O(N)$ 的时间，但由于我们是按倍数（乘法）扩容，扩容不会频繁发生。平摊到每一次操作上，平均运行时间依然被保证为 $O(1)$ ¹⁴。

5. 决定生死的核心：如何设计出色的哈希函数？

扩容机制能保证整体空间足够，但这有个前提：**元素必须能被均匀地分布 (Evenly Distributed)** 到各个桶里。如果哈希函数设计得不好，元素全挤在一个桶里，扩容也没用^{14 15}。

- **避开致命的偶数基数 (Base 126 的教训)**：如果你用 126 作为基数来计算字符串哈希，因为 126 是偶数，在 Java 中 126^{32} 不断相乘并溢出后，**结果会精确地变成 0**。这会导致一个灾难性后果：像 "Laughter is the best thing on the earth." 和 "Muesli is the best thing on the earth." 这样前缀不同、但最后 32 个字符完全一样的句子，计算出的哈希码竟然完全一样（都是 634199182），这将引发极为严重的冲突^{16 17}。
- **采用小素数 (Small Prime)**：为了解决上面的问题，Java 的 `String.hashCode()` 默认选择 31 这个小素数作为乘法基数¹⁷。
 - **为什么是素数**？素数永远不是偶数，多次相乘溢出后不会归零，这打破了上述的魔咒。
 - **为什么是小数字**？小数字计算起来成本更低，速度更快¹⁷。
- **处理负数的终极防线**：计算出的 `hashCode()` 可能是负数。如果你用 `hash % M` 取模来找桶的位置，负数取模在 Java 中依然是负数，会导致数组索引越界报错。因此，最终定位必须使用 `Math.floorMod(hashCode(), M)`，它能确保把任意正负的哈希码安全地转换为 0 到 $M - 1$ 的合法正数索引¹⁸。